Virtual Tables: Creating a View :

View:

it is a virtual table based on a SELECT query.

As you learned earlier, the output of a relational statement such as SELECT is another relation (or table). Suppose that at the end of each day, you would like to have a list of all products to reorder—that is, products with a quantity on hand that is less than or equal to the minimum quantity. Instead of typing the same query at the end of each day, wouldn't it be better to permanently save that query in the database? That is the function of a relational view.

A view is a virtual table based on a SELECT query. The query

can contain columns, computed columns, aliases, and aggregate functions from one or more tables The tables on which the view is based are called base tables.

Based Tables :
the tables on which the view is based .

Creating a view:
Create View viewName AS SELECT query ;

//The CREATE VIEW statement is a data definition command that stores the subquery specification—the SELECT statement used to generate the virtual table—in the data dictionary.

```
SQL> CREATE VIEW PRICEGT50 AS
  2     SELECT P_DESCRIPT, P_QOH, P_PRICE
  3     FROM PRODUCT
  4     WHERE P_PRICE > 50;

View created.

SQL> SELECT * FROM PRICEGT50;

P_DESCRIPT                              P_QOH    P_PRICE
----------------------------------- ---------- ----------
Power painter, 15 psi., 3-nozzle             8     109.99
B&D jigsaw, 12-in. blade                     8     109.92
B&D jigsaw, 8-in. blade                      6      99.87
Hicut chain saw, 16 in.                     11     256.99
Steel matting, 4'x8'x1/6", .5" mesh         18     119.95

SQL> _
```

WWWWOOOOOOOWWWWW:

--> Views are dynamically updated. That is, the view is re-created on demand each time it is invoked. Therefore, if new products are added or deleted to meet the criterion P_PRICE > 50.00, those new products will automatically appear or disappear in the PRICEGT50 view the next time the view is invoked.

--> Views provide a level of security in the database because they can restrict users to seeing only specified columns and rows in a table. For example, if you have a company with hundreds of employees in several departments, you could give each department administrative assistant a view of certain attributes only for the employees who belong to that assistant's department.

--> views can be used as the basis for reports.

the view stores the query and not the relation holding the result , so next time I invoke the view the query within it will run to get the latest data from the tables

Updatable Views:

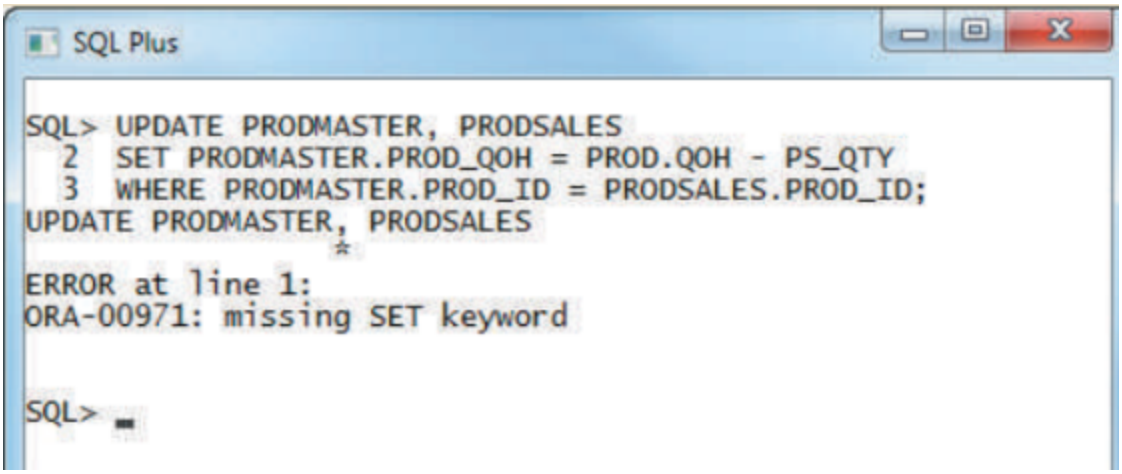
Updatable view is a view that can be used to update attributes in the base tables
• Not all views are updatable
– GROUP BY expressions or aggregate functions cannot be used
– Cannot use set operators
– Most restrictions are based on use of JOINs

One of the most common operations in production database environments is to use batch update routines to update a master table attribute (field) with transaction data. As the name implies, a batch update routine pools multiple transactions into a single batch to update a master table field in a single operation. For example, a batch update routine is commonly used to update a product's quantity on hand based on summary sales transactions. Such routines are typically run as overnight batch jobs to update the quantity on hand of products in inventory. For example, the sales transactions performed by traveling sales people can be entered during periods when the system is offline.

A batch update routine is used to apply many transaction records at once to a master table, typically during off-peak hours, to improve performance and consistency. In the example, monthly product sales stored in the PRODSALES table are subtracted from the inventory quantity stored in the PRODMASTER table. While some DBMSs allow joined UPDATE statements, Oracle does not, so the update must be performed through an updatable view. For a view to be updatable, it must not use aggregates, set operators, or grouping, and the base table to be updated must be key-preserved, meaning its primary key values remain unique in the view. Once such a view is created, updating the view automatically updates the underlying

master table. In practice, however, large-scale batch updates are usually implemented using procedural SQL for better control and reliability.

IN other words , we can say it this way :
Batch update :
it is a routine that applies many transactions at once to a master table .
Often done off-peak to improve performance and consistency.
there is a problem with multi tables updates in Oracle :
oracle does not allow UPDATE with joins directly , so if you need to update a master table based on data from another table , you cannot just write a joined update

- It is **not about updating multiple tables at the same time**.
  - In Oracle, an `UPDATE` statement always updates **one table** at a time.
- The problem is **when the update depends on another table's data**, which usually requires a join to connect the two tables.
- 

- this is the join they are preventing , the fact that you are using more than one table to update the master table is the problem (considred join)

solution : updatable views:

A view can be created on multiple tables , but to be updatable , it must satisfy the following conditions :
1- no aggregate functions
2- no set operators (union , intersect , minus)
3-no **group by** or **distinct**
4-the base table must be key preserved : its primary key must remain unique in the view .

now , when you update the view , Oracle automatically updates the underlying master table .

One easy way to determine whether a view can be used to update a base table is to examine the view's output. If the primary key columns of the base table you want to update still have unique values in the view, the base table is updatable.

you should know that with updatable view , any changes you make to the view are applied directly to the underlying base table .

you have a view:

```
CREATE VIEW v_prod AS
SELECT product_id, qty
FROM prodmaster
WHERE qty > 0;
```

you update the view:

```
UPDATE v_prod
SET qty = qty - 10
WHERE product_id = 101;
```

here oracle translates the update on the view to an update on the base table and only the rows satisfying the condition will be affected .

this is one way to update the master table with data from a transaction table , however the preffered real world solution to the update problem is to use procedural SQL as what will follow .

Oracle Sequences :

in my terms , a sequence is just a generator of numbers , independent of any table , i can use it to fill in a column , all it does is generating numbers , it does not care about where am I gonna put them , so i can use the same sequence and give its values to more than one table .

look , for the other softwares like :
MSAccess ---> if you create a table and forget to define a pk , it will offer to create one for you and it will create a column named ID with an Auto number data type

oracle does not support this , so it traditionally , uses a sequence to assign values to a column on a table .

• Sequences are an independent object in the database. (Sequences are not a data type.)
• Sequences have a name.
• Sequences can be used anywhere a value is expected.
• Sequences are not tied to a table or a column.
• Sequences generate a numeric value that can be assigned to any column in any table.
• The table attribute to which you assigned a value based on a sequence can be edited and modified.

The syntax to create a sequence is as follow:
CREATE SEQUENCE name [START WITH n] [INCREMENT BY n] [CACHE | NOCACHE]

حكاية الكاش و اللا كاش :

In Oracle, sequences have an option called **CACHE** or **NOCACHE**, which controls how the database handles the numbers internally for performance. When you create a sequence with **CACHE**, Oracle preallocates a set of sequence numbers in memory. This means that when your application requests the next number using `NEXTVAL`, Oracle can provide it directly from memory without having to access the disk each time, which makes number generation faster, especially in high-transaction environments. The default cache size is 20 numbers, but it can be increased if needed. On the other hand, using **NOCACHE** tells Oracle **not to preallocate numbers** in memory; each `NEXTVAL` request is generated and stored immediately in the data dictionary on disk. This ensures no numbers are lost in the event of a database crash, but it is slightly slower because each request involves a disk write. Therefore, caching is a **performance optimization**, while NOCACHE prioritizes **consistency and durability** of the sequence values in case of failures.

SO , as Boukhalfa said in the slides:
• MS Access AutoNumber data type fills a
column with unique numeric values
• Oracle sequences
– Independent object in the database
– Named, used anywhere a value expected
– Not tied to a table or column
– Generate numeric values that can be assigned
to any column in any table
– Created and deleted at any time

you can check all your created sequences by the command :

- *select* from USER_SEQUENCES;*

PL / SQL :

SQL does not support conditional execution

- A better approach is to isolate critical code and then have all application programs call the shared code. The advantage of this modular approach is that the application code is isolated in a single program, thus yielding better maintenance and logic control.

To remedy the lack of procedural functionality in SQL and to provide some standardization within the many vendor offerings, the SQL-99 standard defined the use of persistent stored modules. A persistent storage module (PSM) is a block of code containing standard SQL statements and procedural extensions that is stored and executed at the DBMS server. The PSM represents business logic that can be encapsulated, stored,

and shared among multiple database users. A PSM lets an administrator assign specific access rights to a stored module to ensure that only authorized users can use it. Support for PSMs is left to each vendor to implement.

Oracle implements PSMs through its procedural SQL language.

Procedural Language SQL (PL/SQL) is a language that makes it possible to use and store proce-dural code and SQL statements within the database and to merge SQL and traditional programming constructs, such as variables, conditional processing (IF-THEN-ELSE), basic loops (FOR and WHILE loops), and error trapping. The procedural code is exe-cuted as a unit by the DBMS when it is invoked (directly or indirectly) by the end user.
End users can use PL/SQL to create:
• Anonymous PL/SQL blocks
• Triggers (covered in Section 8-7a)
• Stored procedures (covered in Section 8-7b and Section 8-7c)
• PL/SQL functions (covered in Section 8-7d)

PL/SQL BASIC DATA TYPES : (from the slides)
% Type : inherits the data type from a variable that you declared previously or from an attribute of a database table .

Triggers:

A trigger is procedural SQL code that is automatically invoked by the RDBMS upon the occurrence of a given data manipulation event. It is useful to remember that:
• A trigger is invoked before or after a data row is inserted, updated, or deleted.
• A trigger is associated with a database table.
• Each database table may have one or more triggers.
• A trigger is executed as part of the transaction that triggered it.
Triggers are critical to proper database operation and management. For example:
• Triggers can be used to enforce constraints that cannot be enforced at the DBMS design and implementation levels.
• Triggers add functionality by automating critical actions and providing appropriate warnings and suggestions for remedial action. In fact, one of the most common uses for triggers is to facilitate the enforcement of referential integrity.
• Triggers can be used to update table values, insert records in tables, and call other stored procedures.

Triggers play a critical role in making the database truly useful; they also add pro-cessing power to the RDBMS and to the database system as a whole. Oracle recom-mends triggers for:

• Auditing purposes (creating audit logs)
• Automatic generation of derived column values
• Enforcement of business or security constraints
• Creation of replica tables for backup purposes

As you can see, a trigger definition contains the following parts:

• The triggering timing: BEFORE or AFTER. This timing indicates when the trigger's PL/SQL code executes—in this case, before or after the triggering statement is

completed.
• The triggering event: The statement that causes the trigger to execute (INSERT, UPDATE, or DELETE).

– The triggering level: The two types of triggers are statement-level triggers and row-level triggers. A statement-level trigger is assumed if you omit the FOR EACH

ROW keywords. This type of trigger is executed once, before or after the triggering statement is completed. This is the default case.

– A row-level trigger requires use of the FOR EACH ROW keywords. This type of trigger is executed once for each row affected by the triggering statement. (In other words, if you update 10 rows, the trigger executes 10 times.)

IMPORTANT :
The use of the :NEW attribute references deserves a more detailed explanation. To understand its use, you must first consider a basic computing tenet: all changes are done first in primary memory and then transferred to permanent memory. In other words, the computer cannot change anything directly in permanent storage (on disk). It must first read the data from permanent storage to primary memory, then make the change in primary memory, and finally write the changed data back to permanent memory (on disk).
The DBMS operates in the same way, with one addition. Because ensuring data integrity is critical, the DBMS makes two copies of every row being changed by a DML (INSERT, UPDATE, or DELETE) statement. You will learn more about this in Chapter 10. The first copy contains the original ("old") values of the attributes before the changes. The second copy contains the changed ("new") values of the attributes that will be permanently saved to the database after any changes made by an INSERT, UPDATE, or DELETE. You can use :OLD to refer to the original values; you can use :NEW to refer to the changed values (the values that will be stored in the table). You can use :NEW and :OLD attribute references only within the PL/SQL code of a database trigger action.

Although the trigger is a BEFORE trigger, this does not mean that the triggering statement has not executed yet. To the contrary, the triggering statement has already taken place; otherwise, the trigger would not have fired and the :NEW values would not exist. Remember, BEFORE means before the changes are permanently saved to disk, but after the changes are made in memory.
In Oracle, the statement:

```
DROP TRIGGER trigger_name;
```

is used to **permanently delete a trigger** from the database.

Stored Procedures:

A stored procedure is a named collection of procedural and SQL statements. Just like database triggers, stored procedures are stored in the database. One of the major advantages of stored procedures is that they can be used to encapsulate and represent business transactions. For example, you can create a stored procedure to represent a product sale, a credit update, or the addition of a new customer. By doing that, you can encapsulate SQL statements within a single stored procedure and execute them as a single transaction. There are two clear advantages to the use of stored procedures:

• Stored procedures substantially reduce network traffic and increase performance. Because the procedure is stored at the server, there is no transmission of individual SQL statements over the network. The use of stored procedures improves system performance because all transactions are executed locally on the RDBMS, so each SQL statement does not have to travel over the network.

• Stored procedures help reduce code duplication by means of code isolation and code sharing (creating unique PL/SQL modules that are called by application programs), thereby minimizing the chance of errors and the cost of application development and maintenance.

PL/SQL Processing with Cursors:

error. If you want to use a SQL statement that returns
more than one value inside your PL/SQL code, you need to use a cursor. A cursor is a special construct used in procedural SQL to hold the data rows returned by a SQL query. You can think of a cursor as a reserved area of memory in which the output of the query is stored, like an array holding columns and rows. Cursors are held in a reserved memory area in the DBMS server, not in the client computer.

There are two types of cursors: implicit and explicit. An implicit cursor is automatically created in procedural SQL when the SQL statement returns only one value. Up to

this point, all of the examples created an implicit cursor. An explicit cursor is created to hold the output of a SQL statement that may return two or more rows (but could return zero rows or only one). To create an explicit cursor, you use the following syntax inside a PL/SQL DECLARE section:
CURSOR cursor_name IS select-query;

Cursor-style processing involves retrieving data from the cursor one row at a time. Once you open a cursor, it becomes an active data set. That data set contains a “current” row pointer. Therefore, after opening a cursor, the current row is the first row of the cursor.
When you fetch a row from the cursor, the data from the “current” row in the cursor is copied to the PL/SQL variables. After the fetch, the “current” row pointer moves to the next row in the set and continues until it reaches the end of the cursor.

PL/SQL Stored Functions:

• Named group of procedural and SQL
statements that returns a value value, as indicated by a RETURN statement in its program code.

• Syntax:

```
CREATE FUNCTION function_name(argument
IN data-type, ...) RETURN data-type [IS]
BEGIN
PL/SQL statements;
...
RETURN (value or expression);
END;
```

Stored functions can be invoked only from within stored procedures or triggers and cannot be invoked from SQL statements unless the function follows some very specific compliance rules.

• Embedded SQL uses SQL statements within an application programming language